

## PROJECT 2: AURA-db.com

Darell Lewis

Web Development II (Back-End Web Development)

## 1. Introduction

For this project, I wanted to build AURA, which is a website where designers and developers can share cool UI assets like CSS components, color palettes, and SASS mixins. I know the front-end will look nice with modern styling, but I'm focusing on building a solid back-end. Instead of just loading static files, the site will run on a Node.js server using Express to handle routing, and all the asset info will be pulled dynamically from a database.

## 2. Goals and Objectives

My main goal is to connect my first database (MongoDB) to a real website, so it isn't just hardcoded. I also want to learn how to secure the site. That means setting up a login system so users can save their favorite assets and creating a form that checks user input on both the front-end and back-end so nothing bad gets saved to our database. Lastly, I want to make it work offline by making it a Progressive Web App (PWA).

## 3. Target Audience(s)

- **Web Developers:** Looking for copy-paste SASS mixins, responsive layouts, or CSS effects to speed up their projects.
- **UI/UX Designers:** Wanting to share or catalogue custom assets, colors, and patterns.
- **Students & Hobbyists:** Seeking inspiration and free, optimized components for their web applications.

## 4. Key Features

First, there's the Catalog Grid where people can browse and search assets by tags. Second, we'll have User Accounts where passwords get hashed (using bcrypt so they aren't stored in plain text) and sessions keep users logged in. Third, I'm building a Submission Form where creators can upload assets—it'll use Mongoose schemas to validate fields (like checking if image links are actual URLs). Finally, it will have Offline Browsing so it behaves like a mobile app.

## 5. Technology Requirements

- **Front-End UI:** HTML5, SASS, Bootstrap 4, and Vue.js
- **Back-End Runtime:** Node.js and Express.js for the server and routing.
- **Database:** MongoDB (NoSQL)
- **Authentication:** Passport.js or custom express-session middleware with bcrypt
- **Testing:** Mocha/Chai
- **Security & SSL:** HTTPS configuration and HTTP-only cookies to protect session IDs.